

Table of Contents

Library Design and Architecture Choices	2
Project structure.....	2
Layer and activation design	2
Loss and gradient checking	3
XOR test output	4
Github Link:.....	4

Library Design and Architecture Choices

In this project we implemented a small neural-network library from scratch using only NumPy.

Our main goal was to separate responsibilities into clear modules so that the same code base can be reused for simple problems such as XOR and also for more complex models such as the MNIST autoencoder and the latent-space SVM.

Project structure

The library is organized in a Python package called **lib/**:

- **lib/layers.py** – base Layer class and the fully connected Dense layer.
- **lib/activations.py** – non-linear activation layers (ReLU, Sigmoid, Tanh).
- **lib/losses.py** – loss functions and their gradients (mean squared error in our case).
- **lib/network.py** – the Sequential model that stores the layers and runs training.
- **lib/optimizer.py** – placeholder for optimizers (we use simple SGD style updates).

Outside the library we have:

- **notebooks/** – Jupyter notebooks for gradient checking, XOR, autoencoder and SVM experiments.
- **report/** – the written report (project_report.pdf).
- **requirements.txt** – lists the Python dependencies for reproducibility.

This separation makes the **library code** independent from the **experiments and report**, similar to how real ML projects are structured.

Layer and activation design

All layers inherit from a simple abstract Layer class that defines two methods:

- **forward(input)** – computes the output of the layer.
- **backward(grad_output, learning_rate)** – propagates gradients and updates parameters.

The Dense layer implements a fully connected transformation

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

and stores the input so that it can compute gradients during backpropagation.

Activation functions (Tanh, Sigmoid, ReLU) are implemented as separate layers.

This design lets us build different architectures just by changing the sequence of layers, for example:

- **XOR network:** Dense → Tanh → Dense → Tanh
- **Autoencoder:** Dense → Tanh → ... → Dense → Tanh

Keeping activations as layers also makes the backward pass cleaner, because each activation knows its own derivative.

Loss and gradient checking

The mean squared error loss (`mse_loss`) and its analytical gradient (`mse_grad`) are implemented in `losses.py`.

During training the `Sequential` model calls the loss function and then uses its gradient as the starting point for backpropagation.

To verify that our backpropagation implementation is correct, we added **numerical gradient checking** in the notebook.

The `Dense` layer stores its parameter gradients (`dW`, `db`) during the backward pass so we can compare them with numerical gradients computed with

XOR test output

```
... Epoch 0, Loss = 1.3072276031160035
Epoch 500, Loss = 1.000144700440011
Epoch 1000, Loss = 0.9999725832468229
Epoch 1500, Loss = 0.9994381641668579
Epoch 2000, Loss = 0.0015129328725747957
Epoch 2500, Loss = 0.0005706018952434804
Epoch 3000, Loss = 0.00034355288980721446
Epoch 3500, Loss = 0.00024352552982659248
Epoch 4000, Loss = 0.00018770058062239642
Epoch 4500, Loss = 0.00015224478453135628
Epoch 5000, Loss = 0.00012780096568198218
Epoch 5500, Loss = 0.0001099639575084387
Epoch 6000, Loss = 9.639360488816856e-05
Epoch 6500, Loss = 8.57341526401248e-05
Epoch 7000, Loss = 7.714701622249367e-05
Epoch 7500, Loss = 7.008621643880083e-05
Epoch 8000, Loss = 6.418130075057737e-05
Epoch 8500, Loss = 5.9172226438571244e-05
Epoch 9000, Loss = 5.4871182977813874e-05
Epoch 9500, Loss = 5.113920473586588e-05
Epoch 10000, Loss = 4.787130240280181e-05
Epoch 10500, Loss = 4.498670461656956e-05
Epoch 11000, Loss = 4.242227425400088e-05
Epoch 11500, Loss = 4.0127957918901506e-05
Epoch 12000, Loss = 3.806357286294065e-05
...
Final predictions:
[[-0.99663606  0.99500519  0.99518517 -0.99516401]]
True labels:
[[-1.  1.  1. -1.]]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK

Github Link:

https://github.com/MonMon204/CI_Project.git